

Lab 2: PHY-Layer Simulation Using GNURadio

Author: Andrew J Napieraj

Class: ESCE 6660 Internetworking of Things (IoT)

Instructor: Prof. Ish Jain

TA: Chibuikem Ezemaduka

Due Date: February 14, 2026

Description

This lab explores the use of the GNURadio application, and the implementation of Binary Phase Shift Key (BPSK), and Quadrature Phase Shift Key (QPSK) modulation schemes within a prebuilt lab environment. For ECSE 6660 course, we will additionally explore the use of Gray codes and discuss why they improve Bit-Error Rate (BER) overall within a modulation scheme.

Part 1 of this lab will focus on setup steps to include some difficulties from initial setup, lessons learned, and tools acquired from the conduct of the lab. Additionally, we will include screen captures of the minor (aesthetic and flexibility) changes to the data flow graphs (DFGs). These changes do not change the overall functionality of the source code.

Part 2 will include requisite BER tables and semilogy graphs of BER in the various modulation schemes. We will also answer all the lab questions proposed in the Lab assignment sheet.

The appendix will contain the code blocks for the various modulation schemes, which will also be available via the GitHub link provided in the materials list.

Materials

- [lot Virtualbox](#)
- [Source Code](#) for out of tree (OOT) module
- [BPSK Source Code](#)
- [QPSK Source Code](#)
- [Gnuradio](#) GUI 3.10.1.1
- [Modulation Scheme Codex \(GitHub\)](#)

Part 1: Lab setup, difficulties, and lessons learned

Overall, the setup of this lab was very straightforward and easy to accomplish due to the sufficient training gained from the first lab. A minor sticking point was initially using the Gradescope copy of the lab documentation, which has the incorrect link for the QPSK source code, but this was easily corrected by simply going to the Piazza version of the lab documentation, which has been adequately discussed as the source information for the course.

From initial install to setup, I made some minor adjustments to the DFG to more quickly run through the lab without having to enable or disable blocks as I troubleshoot code modulation schemes. *Figure 1*, *Figure 2*, and *Figure 3* correspond to BPSK, QPSK, and QPSK (Gray) respectively. For *Figure 3*, we see the ground truth block was removed for the Gray-coded modulation due to it using index ordering [0,1,3,2] instead of the standard [0,1,2,3], which initially produced artificially high BER from the given function.

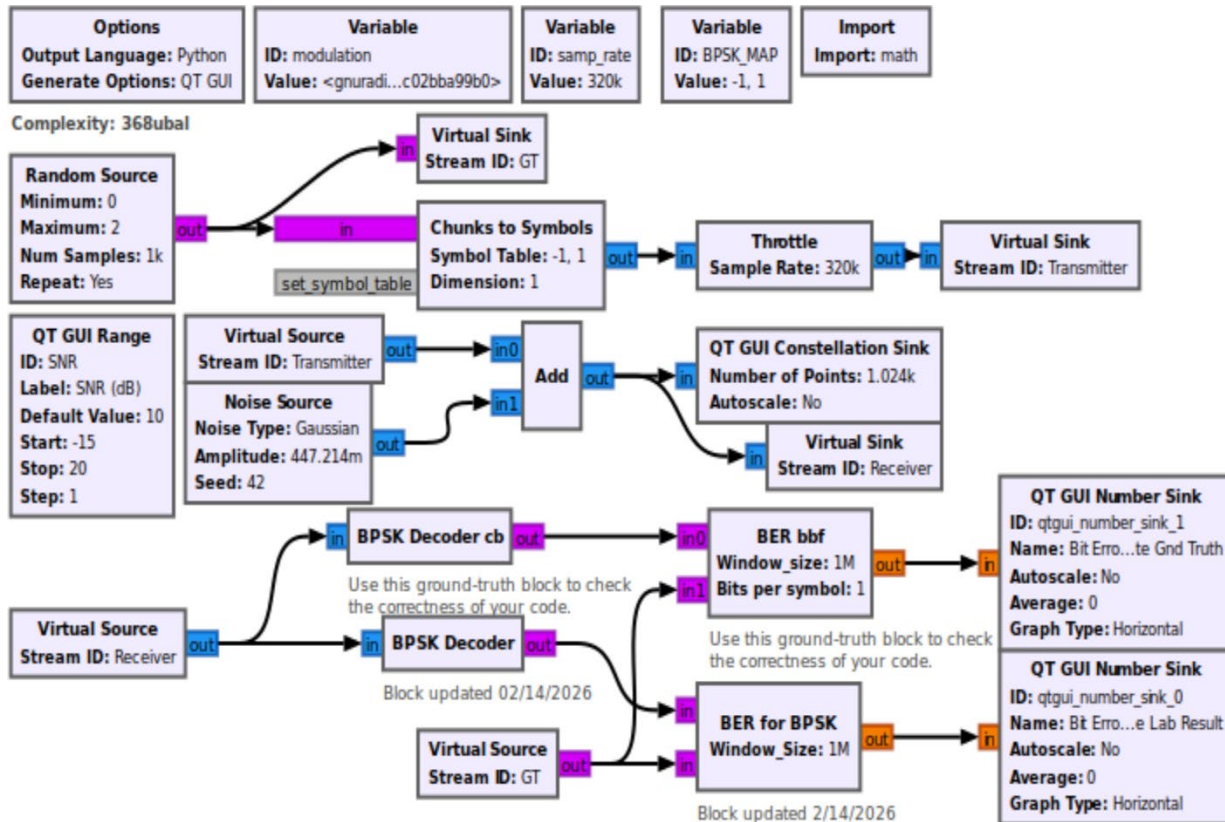


Figure 1: BPSK DFG

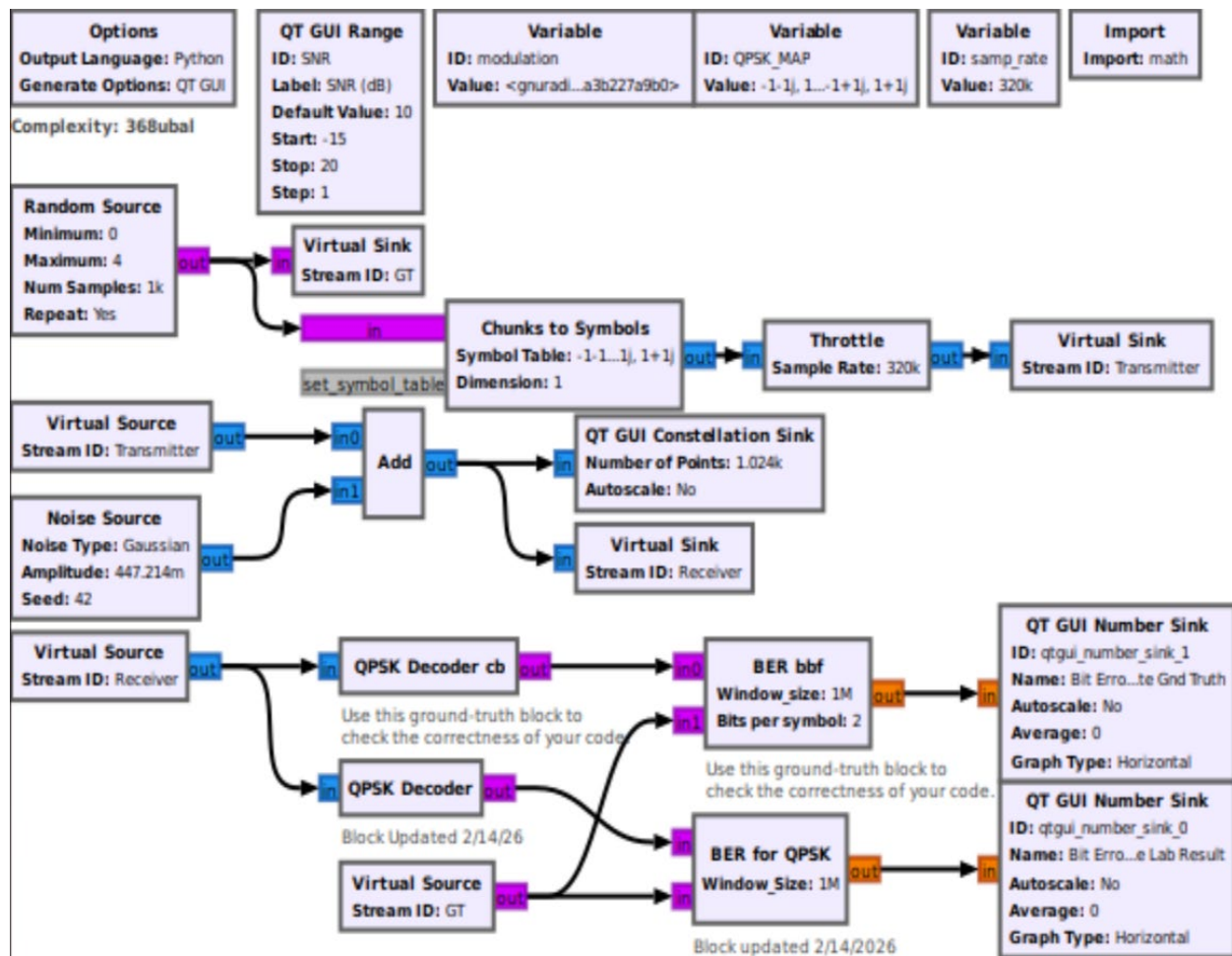


Figure 2: QPSK DFG

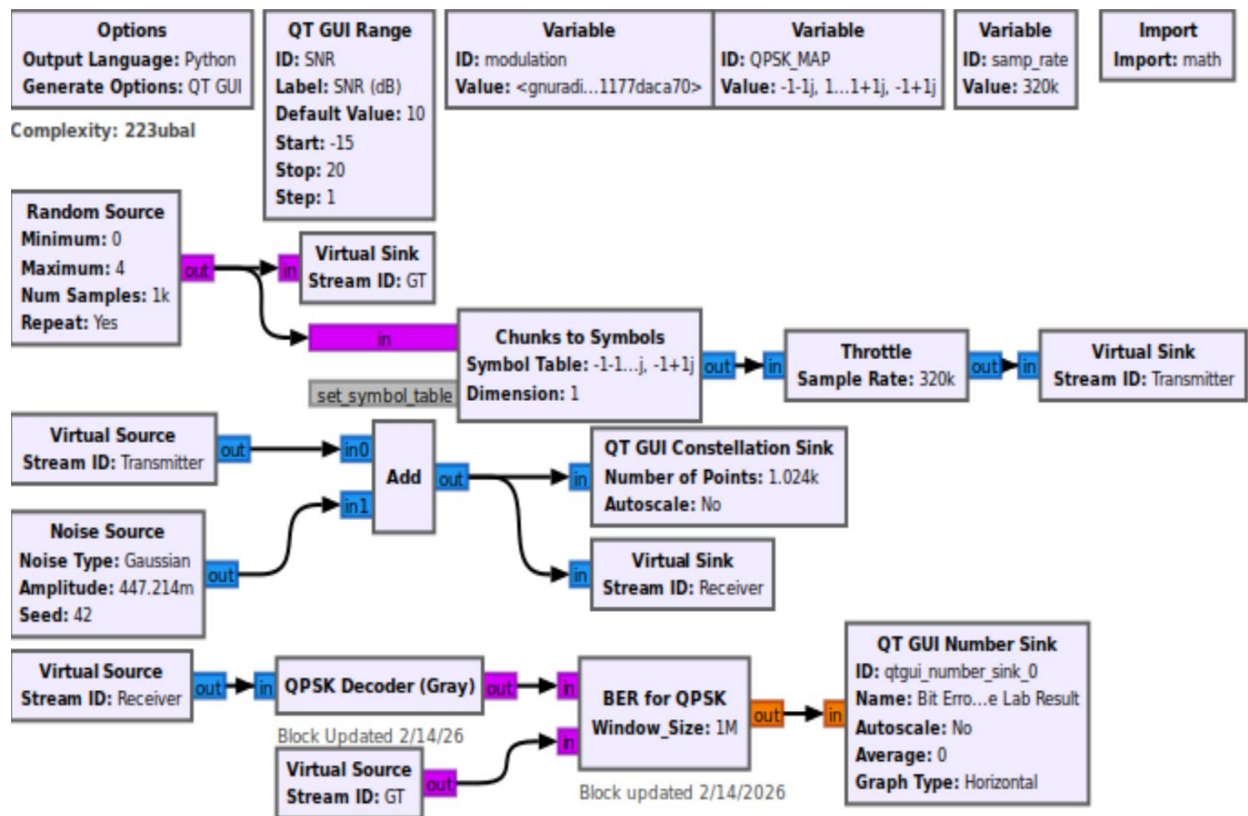


Figure 3: QPSK (Gray) DFG

Part 2: Deliverables and lab execution

Effect of SNR on BPSK/QPSK

Decreasing SNR: noise variance increases, so received constellation points spread out into “clouds” around the ideal symbol locations. At low SNR, clouds overlap decision boundaries → frequent symbol/bit errors.

Increasing SNR: noise variance decreases, so the clusters tighten around the ideal points. At high SNR, clusters are well separated → very low error probability.

QPSK constellation (complex plane): four points at $(\pm 1, \pm 1)$ (up to any scaling) as seen in Figure 4 and Table 1.

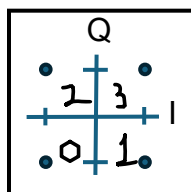


Figure 4: QPSK Constellation Diagram

Modulation	Data Bit(s)	Symbol
BPSK	0	-1
	1	1
QPSK	00	-1-j
	01	1-j
	10	-1+j
	11	1+j

Table 1: BPSK and QPSK Mapping

BPSK decoding (hard decision)

The BPSK symbols lie on the real axis. Decide based on the sign of the real part:

$$\hat{s} = \begin{cases} 1, & \Re\{r\} \geq 0 \\ 0, & \Re\{r\} < 0 \end{cases}$$

QPSK decoding (hard decision)

Decide the quadrant from the sign of I and Q:

$$I = \Re\{r\}, Q = \Im\{r\}$$

$$\hat{I} = 1[I \geq 0], \hat{Q} = 1[Q \geq 0]$$

Then map quadrant $\rightarrow$ symbol ID (0–3) consistent with the lab's QPSK_MAP.

QPSK BER computation (bit-wise)

Even if two QPSK symbols differ, they may differ by 1 bit or 2 bits. Compute:

- $e = a \oplus b$ on the 2 LSBs
- bit errors per symbol = popcount(e)
- BER (per bit) = average(bit errors) / 2

QPSK (Gray) Implementation

The modified QPSK_MAP produces a different labeling of the same four points (still at $(\pm 1, \pm 1)$), but the bit assignments around the square change as seen in *Figure 5* and *Table 2*.

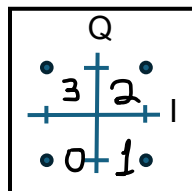


Figure 5: QPSK (Gray) Constellation Diagram

Modulation	Data Bit(s)	Symbol
QPSK(Gray)	00	-1-j
	01	1-j
	10	1+j
	11	-1+j

Table 2: QPSK (Gray-coded) Mapping

The Gray-coded mapping has lower BER than a non-gray mapping at the same SNR, because most symbol errors at moderate/high SNR are nearest-neighbor errors; Gray coding ensures nearest neighbors differ by only 1 bit, so a symbol error typically causes 1 bit error instead of 2-bit errors.

Observed BER Trend

From the BER tables collected for BPSK, QPSK, and QPSK (Gray), it is observed that the BER decreases monotonically as SNR increases. At low SNR values (-15 dB to -6 dB), the constellation points are highly dispersed due to noise, resulting in frequent decision boundary crossings and high BER values. As SNR increases toward positive values, the constellation clusters tighten around their ideal locations, reducing symbol misclassification and lowering BER rapidly.

The QPSK (non-Gray) modulation exhibits significantly higher BER than both BPSK and Gray-coded QPSK at the same SNR. This is because nearest-neighbor symbol errors in the non-Gray mapping often produce two bit errors instead of one. In contrast, Gray-coded QPSK ensures adjacent constellation points differ by only one bit, so most symbol errors result in a single bit error. As SNR increases, all curves begin to converge toward zero, but the Gray-coded QPSK curve consistently remains below the non-Gray QPSK curve.

BER vs SNR Plot

Figure 6 shows the BER vs SNR curves plotted on a logarithmic scale using the semilogy function. The logarithmic scale clearly illustrates the exponential decay of BER with increasing SNR. The BPSK and Gray-coded QPSK curves demonstrate similar slopes, while the non-Gray QPSK curve remains consistently higher across the

measured SNR range.

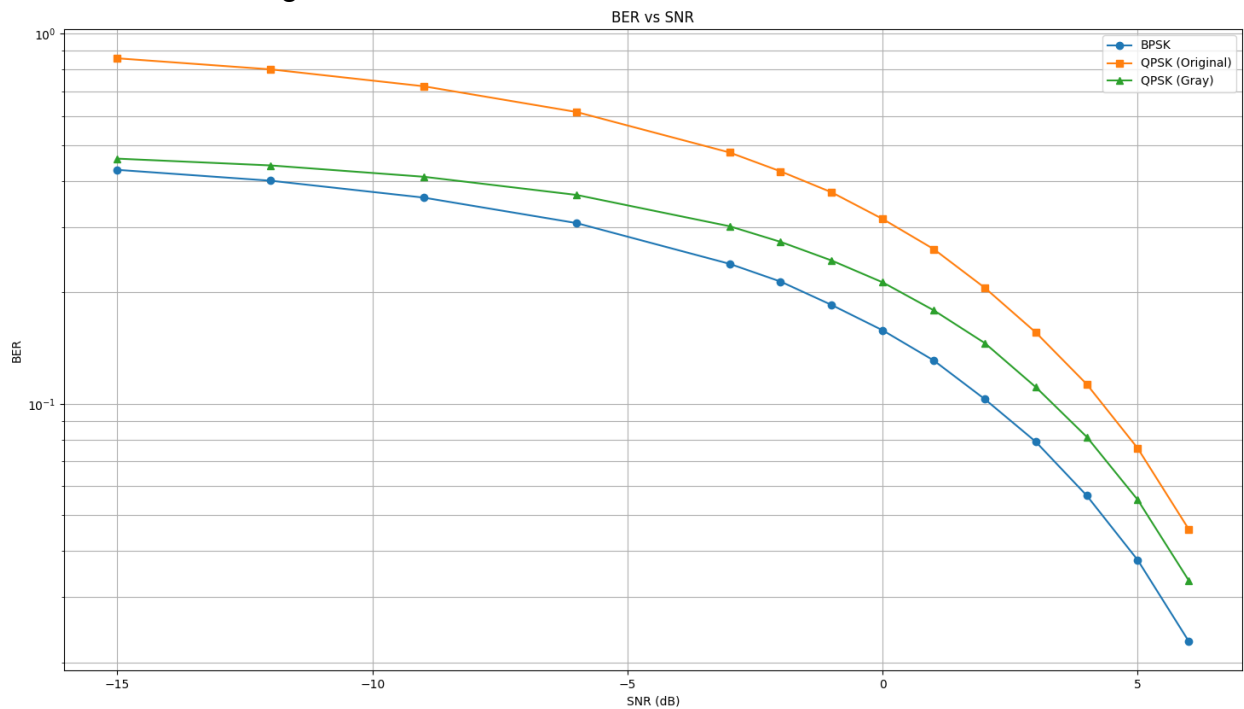


Figure 6: SNR vs. BER for all modulation schemes

SNR Threshold for Reliable Decoding

By manually increasing the SNR in the GNU Radio GUI beyond the tabulated range and observing the BER values, the approximate SNR threshold beyond which $\text{BER} < 1 \times 10^{-4}$ was determined.

For BPSK modulation, the BER dropped below 1×10^{-4} at approximately:

SNR ≈ 11.5 – 12.0 dB

For Gray-coded QPSK, the BER also dropped below 1×10^{-4} at approximately:

SNR ≈ 11.5 – 12.0 dB

For non-Gray QPSK, a slightly higher SNR was required to reach the same BER threshold due to the increased average number of bit errors per symbol, approximately:

SNR ≈ 12.0 – 12.5 dB

These results are consistent with theoretical expectations, as both BPSK and Gray-coded QPSK have identical theoretical BER performance under equal energy-per-bit conditions in AWGN channels.

Comparison of BPSK and QPSK BER vs SNR

Under additive white Gaussian noise (AWGN) and coherent detection, BPSK and Gray-coded QPSK exhibit identical theoretical BER performance when normalized by energy per bit (E_b/N_0). This is because QPSK can be interpreted as two independent orthogonal BPSK systems operating on the in-phase (I) and quadrature (Q) components.

The theoretical BER expression for both modulations is:

$$P_b = Q(\sqrt{2E_b/N_0})$$

Therefore, when plotted versus E_b/N_0 , the BER vs SNR curves for BPSK and Gray-coded QPSK should coincide.

However, if SNR is defined per symbol energy (E_s/N_0) rather than per bit energy, the curves may appear shifted due to QPSK transmitting two bits per symbol.

Conclusion

This lab demonstrated the implementation and analysis of BPSK and QPSK modulation schemes within GNU Radio. Through constellation analysis and BER measurements, it was observed that increasing SNR reduces constellation dispersion and significantly lowers BER. Gray coding was shown to improve QPSK performance by minimizing bit errors resulting from nearest-neighbor symbol mistakes. Experimental results aligned with theoretical expectations, confirming that BPSK and Gray-coded QPSK achieve similar BER performance under equal energy-per-bit conditions.

Acknowledgements

This lab was completed using GNU Radio 3.10.1 within the provided Ubuntu 22.04 VirtualBox environment. The out-of-tree (OOT) module supplied by the course staff was used to validate BER measurements and ensure proper modulation mapping.

Official GNU Radio documentation and the course lab manual were referenced throughout the implementation and debugging process.

AI-assisted tools were used for troubleshooting Python script errors, verifying GNU Radio block behavior, and refining the clarity and organization of this written report. All simulation design decisions, modulation mapping implementations, and BER analysis were independently implemented and verified by the author.

References

- [1] GNU Radio Wiki, "What is GNU Radio?", Available:
https://wiki.gnuradio.org/index.php/What_is_GNU_Radio%3F
- [2] GNU Radio Documentation (Doxygen), Available:
<https://www.gnuradio.org/doc/doxygen/>
- [3] Lab 2: PHY-Layer Simulation Using GNU Radio, ESCE 4660/6660, Rensselaer Polytechnic Institute, Spring 2026

Appendix

BPSK BER Values

SNR (dB)	BER for BPSK
-15	0.429
-12	0.401
-9	0.361
-6	0.308
-3	0.239
-2	0.214
-1	0.185
0	0.158
1	0.131
2	0.103
3	0.0790
4	0.0565
5	0.0378
6	0.0228

BPSK Decoding Block

```
import numpy as np
from gnuradio import gr

class blk(gr.sync_block):
    def __init__(self):
        gr.sync_block.__init__(
            self,
            name='BPSK Decoder',
            in_sig=[np.complex64],
            out_sig=[np.uint8]
        )

    def work(self, input_items, output_items):
        x = input_items[0]
        y = output_items[0]

        # If real>=0 -> symbol 1, else symbol 0
        y[:] = (np.real(x) >= 0).astype(np.uint8)

        return len(output_items[0])
```

QPSK BER Values

SNR (dB)	BER for QPSK
-15	0.859
-12	0.802
-9	0.722
-6	0.615
-3	0.478
-2	0.425
-1	0.373
0	0.316
1	0.262
2	0.206
3	0.156
4	0.113
5	0.0758
6	0.0458

QPSK Decoding Block

```
import numpy as np
from gnuradio import gr

class blk(gr.sync_block):
    def __init__(self):
        gr.sync_block.__init__(
            self,
            name='QPSK Decoder',
            in_sig=[np.complex64],
            out_sig=[np.uint8]
        )

    def work(self, input_items, output_items):
        x = input_items[0]
        y = output_items[0]

        I = (np.real(x) >= 0).astype(np.uint8)
        Q = (np.imag(x) >= 0).astype(np.uint8)

        # quad code: BL=0, BR=1, TL=2, TR=3
        quad = (Q << 1) | I

        # map quad -> QPSK_MAP index: BL->0, BR->1, TL->2, TR->3
        lut = np.array([0, 1, 2, 3], dtype=np.uint8)
        y[:] = lut[quad]

        return len(output_items[0])
```

QPSK (Gray) BER Values

SNR (dB)	BER for QPSK (Gray)
-15	0.460
-12	0.441
-9	0.411
-6	0.367
-3	0.302
-2	0.274
-1	0.244
0	0.213
1	0.179
2	0.146
3	0.111
4	0.0814
5	0.0550
6	0.0333

QPSK (Gray) Decoding Block

```
import numpy as np
from gnuradio import gr

class blk(gr.sync_block):
    def __init__(self):
        gr.sync_block.__init__(
            self,
            name='QPSK Decoder (Gray)',
            in_sig=[np.complex64],
            out_sig=[np.uint8]
        )

    def work(self, input_items, output_items):
        x = input_items[0]
        y = output_items[0]

        I = (np.real(x) >= 0).astype(np.uint8)
        Q = (np.imag(x) >= 0).astype(np.uint8)

        # quad code: BL=0, BR=1, TL=3, TR=2
        quad = (Q << 1) | I

        # map quad -> QPSK_MAP index: BL->0, BR->1, TL->3, TR->2
        lut = np.array([0, 1, 3, 2], dtype=np.uint8)
        y[:] = lut[quad]

        return len(output_items[0])
```

BPSK BER Calculation

```
import numpy as np
from gnuradio import gr

class blk(gr.sync_block):
    def __init__(self, window_size=100000):
        gr.sync_block.__init__(
            self,
            name='BER for BPSK',
            in_sig=[np.byte, np.byte],
            out_sig=[np.float32]
        )
        self.window_size = int(window_size)
        self.window = np.zeros(self.window_size,
dtype=np.float32)
        self.win_ptr = 0

    def work(self, input_items, output_items):
        ref = input_items[0].astype(np.int8)
        rx = input_items[1].astype(np.int8)

        n = min(len(ref), len(rx))

        # For BPSK bits: error per symbol is XOR (0 or 1)
        errs = (ref[:n] ^ rx[:n]).astype(np.float32)

        # cyclic queue update
        for k in range(n):
            self.window[self.win_ptr] = errs[k]
            self.win_ptr = (self.win_ptr + 1) % self.window_size

        BER = float(np.sum(self.window)) /
float(self.window_size)

        # output BER for every produced sample (simple +
compatible with sinks)
        output_items[0][:n] = BER

        return len(output_items[0])
```


QPSK BER Calculation

```
import numpy as np
from gnuradio import gr

class blk(gr.sync_block):

    def __init__(self, window_size=100000):
        gr.sync_block.__init__(
            self,
            name='BER for QPSK',
            in_sig=[np.byte, np.byte],
            out_sig=[np.float32]
        )

        self.window_size = int(window_size)
        self.window = np.zeros(self.window_size,
                                dtype=np.float32)
        self.win_ptr = 0

    def work(self, input_items, output_items):

        a = input_items[0].astype(np.uint8)
        b = input_items[1].astype(np.uint8)

        # sync_block guarantees same input lengths
        n = len(a)

        # XOR to find bit mismatches
        err = (a ^ b) & 0x03

        # Count bit errors (0,1,2)
        biterrors = ((err & 0x01) + ((err & 0x02) >>
1)).astype(np.float32)

        # Update cyclic window
        for k in range(n):
            self.window[self.win_ptr] = biterrors[k]
            self.win_ptr = (self.win_ptr + 1) % self.window_size

        # True BER per bit (2 bits per QPSK symbol)
        BER = np.sum(self.window) / (2*self.window_size)

        # Must output exactly n samples
        output_items[0][:] = BER

        return len(output_items[0])
```